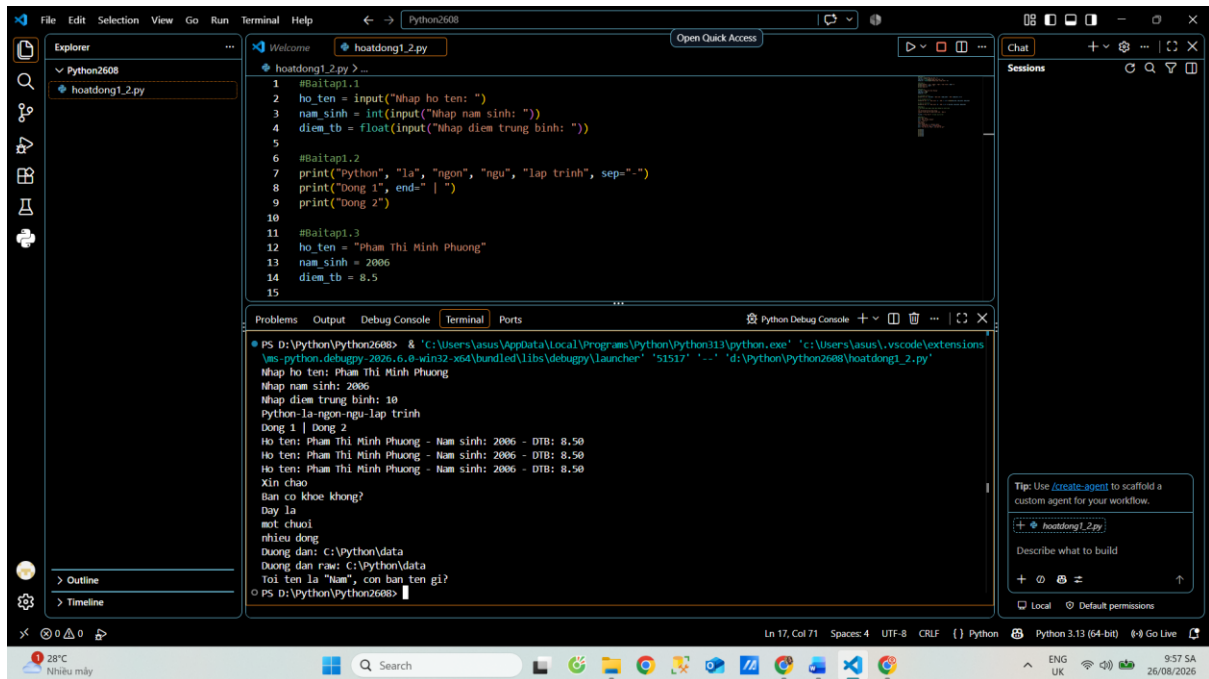


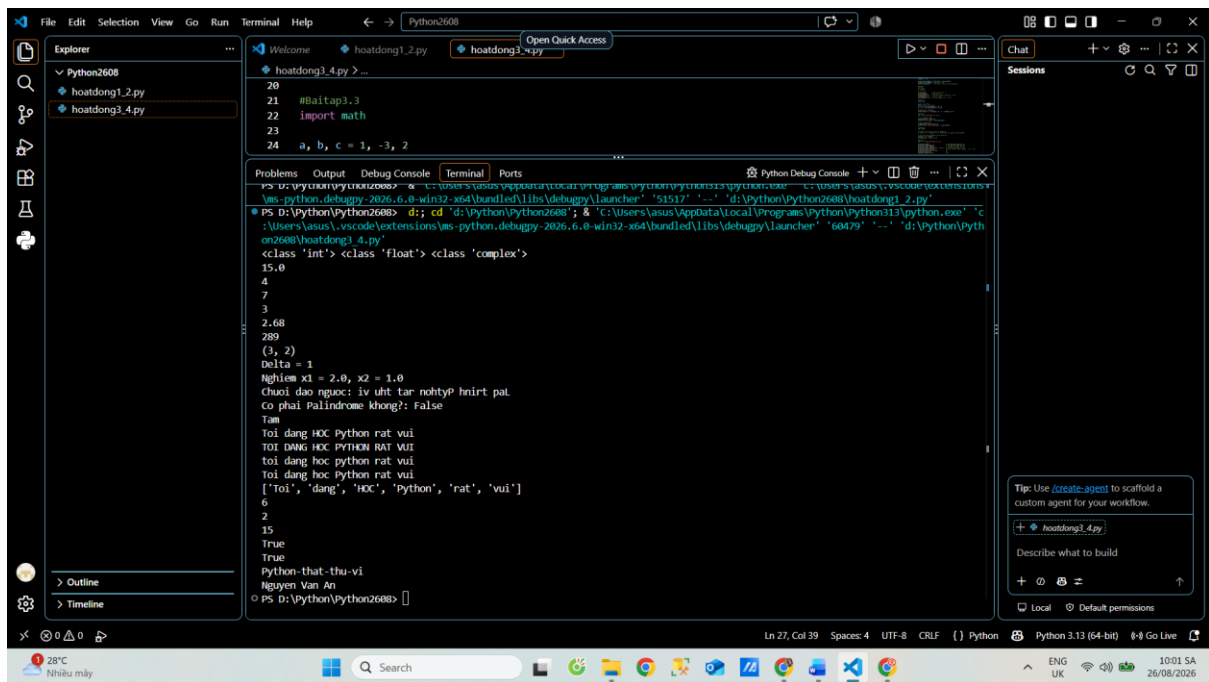
Hoạt động 1_2



```
1 #Baitap1.1
2 ho_ten = input("Nhap ho ten: ")
3 nam_sinh = int(input("Nhap nam sinh: "))
4 diem_tb = float(input("Nhap diem trung binh: "))
5
6 #Baitap1.2
7 print("Python", "la", "ngon", "ngu", "lap trinh", sep="-")
8 print("Dong 1", end=" | ")
9 print("Dong 2")
10
11 #Baitap1.3
12 ho_ten = "Pham Thi Minh Phuong"
13 nam_sinh = 2006
14 diem_tb = 8.5
15
```

```
PS D:\Python\Python2608> & 'C:\Users\sasus\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\sasus\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '51517' '-' 'd:\Python\Python2608\hoatdong1_2.py'
Nhap ho ten: Pham Thi Minh Phuong
Nhap nam sinh: 2006
Nhap diem trung binh: 8.5
Python-la-ngon-ngu-lap-trinh
Dong 1 | Dong 2
Ho ten: Pham Thi Minh Phuong - Nam sinh: 2006 - DTB: 8.50
Ho ten: Pham Thi Minh Phuong - Nam sinh: 2006 - DTB: 8.50
Ho ten: Pham Thi Minh Phuong - Nam sinh: 2006 - DTB: 8.50
Xin chao
Ban co khoe khong?
Day la
mot chuoai
nhiều dòng
Dung dan: c:\python\data
Dung dan raw: c:\python\data
toi ten la "Nam", con ban ten gi?
PS D:\Python\Python2608>
```

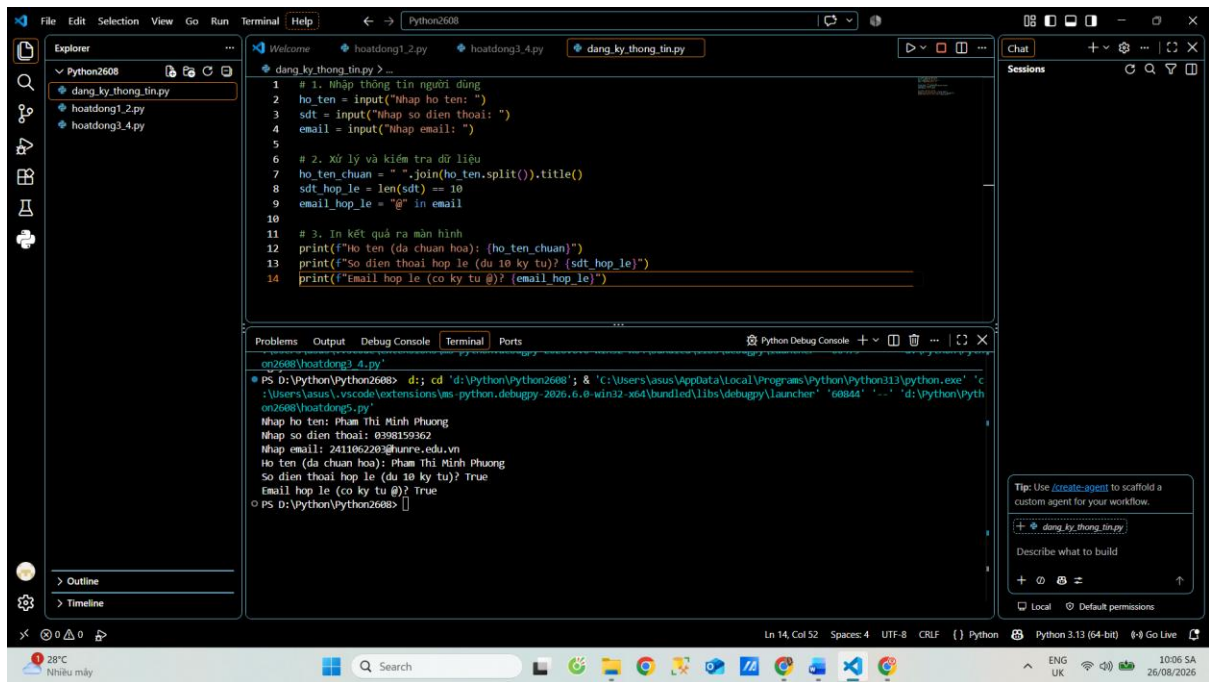
Hoạt động 3_4



```
20
21 #Baitap3.3
22 import math
23
24 a, b, c = 1, -3, 2
```

```
PS D:\Python\Python2608> & 'C:\Users\sasus\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\sasus\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '51517' '-' 'd:\Python\Python2608\hoatdong1_2.py'
PS D:\Python\Python2608> cd 'd:\Python\Python2608'; & 'C:\Users\sasus\AppData\Local\Programs\Python\Python313\python.exe' 'c:\Users\sasus\.vscode\extensions\ms-python.debugpy-2026.6.0-win32-x64\bundle\libs\debugpy\launcher' '60479' '-' 'd:\Python\Python2608\hoatdong3_4.py'
<class 'int'> <class 'float'> <class 'complex'>
15.0
4
7
3
2.68
709
(3, 2)
Delta = 1
Nghiem x1 = 2.0, x2 = 1.0
Chuoai dao nguoc: iv uht tar nohtyP hnirt pal
Co phai Palindrome khong?: False
Tam
Toi dang hoc Python rat vui
TOI DAMG HOC PYTHON RAT VUI
toi dang hoc python rat vui
Toi dang hoc python rat vui
['Toi', 'dang', 'hoc', 'Python', 'rat', 'vui']
6
2
15
True
True
Python-that-thu-vi
Nguyen Van An
PS D:\Python\Python2608>
```

Hoạt động 5



Hoạt động 1

Bài 1.1

ho_ten không cần ép kiểu: Hàm input() mặc định thu về chữ (chuỗi). Tên người vốn dĩ là chữ sẵn rồi nên giữ nguyên, không cần biến đổi gì nữa.

nam_sinh và diem_tb phải ép kiểu: Khi nhập số qua input(), Python vẫn hiểu đó là chữ (ví dụ nhập 2004 thì nó hiểu là chuỗi ký tự "2004" chứ không phải số). Dùng int() để đổi thành số nguyên và float() để đổi thành số thập phân. Phải đổi sang kiểu số thì sau này mới đem cộng trừ, tính tuổi hay tính điểm trung bình được.

Bài 1.2

Với sep=", ": Nó sẽ chèn một dấu phẩy và một dấu cách vào giữa mỗi từ.

Dấu nhảy đơn “\n”: nó sẽ kẹp một dấu nhảy đơn vào giữa các chữ.

Dùng sep="\n" (xuống dòng): Cứ in xong một từ thì nó tự bấm Enter xuống dòng rồi mới in từ tiếp theo.

Bài 1.3

Viết nhanh, dễ nhìn: Biến nằm ở đâu thì nhét thẳng vào chỗ đó trong câu luôn. Hai cách còn lại phải để biến ở tận cuối câu, lúc đọc code cứ phải đóng mắt từ trong ra ngoài rất đau mắt.

Chạy nhanh hơn: f-string được Python tối ưu tốt hơn nên máy xử lý nhẹ và nhanh hơn hai kiểu cũ.

Tiện lợi: Có thể tính toán hoặc xử lý nhanh gọn ngay trong dấu {} luôn mà không cần tạo thêm biến phụ.

Hoạt động 2

Khác biệt giữa s4 và s5:

Cách viết:

s4 = "Duong dan: C:\\Python\\data": Phải viết 2 dấu gạch chéo \\ để Python hiểu là muốn in ra 1 dấu \.

s5 = r"Duong dan: C:\Python\data": Thêm chữ r ở đầu (viết tắt của *raw* - thô/nguyên bản), giúp gõ trực tiếp 1 dấu \ mà không bị lỗi.

Kết quả in ra màn hình: Cả hai dòng cho ra kết quả giống hệt nhau: Duong dan: C:\Python\data.

Raw string (r"...") dùng khi nào?

Nó dùng khi trong chuỗi có chứa nhiều dấu gạch chéo ngược \, giúp bạn viết thẳng dấu \ mà không cần phải gõ hai lần \\ để "né" lỗi.

Đường dẫn thư mục (Windows): r"C:\Users\Admin\Desktop\folder" (gõ nhanh hơn hẳn việc viết \\).

Biểu thức chính quy (Regex): Rất hay dùng để tìm kiếm chuỗi, nơi sử dụng cực kỳ nhiều dấu \.

Hoạt động 3

So sánh pow(c, 2) và c ** 2:

Kết quả: Hai cách này không phải luôn luôn cho kết quả giống nhau trong mọi trường hợp.

Vì sao?

Khi chỉ dùng 2 tham số (như pow(c, 2) và c ** 2): Cả 2 cho kết quả giống hệt nhau (đều tính c^2).

Điểm khác biệt lớn nhất: Hàm pow() có thể nhận 3 tham số, còn toán tử ** thì không:

Ví dụ: `pow(c, 2, 5)` sẽ tính $(c^2) \pmod{5}$ (tức là lấy c^2 rồi chia cho 5 lấy phần dư). Cách này tính toán cực kỳ nhanh khi làm việc với số lớn (hay dùng trong mật mã học).

Viết theo kiểu toán tử tương đương sẽ là `(c ** 2) % 5`, nhưng máy tính phải tính ra kết quả c^2 trước rồi mới đem chia, tốn bộ nhớ và chậm hơn rất nhiều nếu số cực bự.

<https://github.com/2411062203-create/Python2608>